
A Supervisor Multi-Agent Graph in LangGraph

Researcher, writer and critic under an LLM router — design, full source, and the two termination bugs the tests caught.

Created: Monday, 31 August 2026, 8:12 PM PDT

Source file: `langgraph_supervisor_multi_agent.py`

Test file: `langgraph_supervisor_multi_agent_wiring_test.py`

Location: `~/Documents/Code_Projects/AI_study/learning/codes`

Stack: langgraph 0.6.11 · langchain-openai 0.3.35 · openai gpt-4o-mini · ddgs 9.8 · pydantic 2.13 · Python 3.12

Index

1	What this document covers	3
2	The design, and what changed from the first sketch	3
2.1	The original plan	3
2.2	Four corrections	3
3	Graph topology	4
4	The shared state object	5
4.1	Field-by-field	5
4.2	Reducers, and why they matter	6
5	Agent-to-agent message passing	6
6	The supervisor router	7
6.1	Structured output, not free text	7
6.2	The routing prompt	8
7	Routing guards	9
7.1	Why guards, and what they may not do	9
7.2	Fixed-point iteration	11
8	Termination: two budgets	12
8.1	The revision budget	12
8.2	The critic loop — a bug the tests caught	12
8.3	recursion_limit is not termination logic	13
9	The three specialists	14
10	Assembling and running the graph	16
11	Test results	17
12	How to run it	18
13	References	19

1. What this document covers

This is the design and implementation of a three-specialist agent team orchestrated by a LangGraph supervisor. A **researcher** searches the web, a **writer** turns the findings into a blog post, and a **critic** reviews the result. A **router** node decides who runs next, using an LLM call rather than a hard-coded sequence.

The interesting part is not the happy path — three agents in a row is easy. It is what happens when the LLM router makes a bad call, and what stops the graph running forever when the critic is never satisfied. Two real bugs of exactly that kind were found by the test harness while this code was being written; both are documented in section 8, because the reasoning behind them is more valuable than the fix.

2. The design, and what changed from the first sketch

2.1 The original plan

The starting sketch was a straight-line pipeline with the router in the middle:

the first sketch

```
state: topic, research, draft, comment

START    -> router
router   -> researcher (passes state["topic"])
researcher -> router   (returns state["research"])
router   -> writer     (passes state["topic"], state["research"])
writer   -> router     (returns state["draft"])
router   -> critic     (passes topic, research, draft)
critic   -> router     (returns state["comment"])

final output = state["comment"]
```

The overall shape — a supervisor with specialists reporting back — is the right one, and it survives into the final code. Four things needed changing.

2.2 Four corrections

Issue	In the sketch	In the implementation
Edges do not carry fields	router -> researcher (passes state["topic"])	Every node receives the <i>whole</i> state and returns a partial dict that LangGraph merges. The researcher reads state["topic"] itself. There is no plumbing to build for this.

Issue	In the sketch	In the implementation
The router had nothing to decide	research → draft → comment is fixed, so an LLM call only discovers which slot is empty — a slow, non-deterministic if.	The critic may send work back to the writer. The router now chooses between <i>revise</i> and <i>ship</i> , which is a judgment an LLM is actually suited to.
Wrong final output	state["comment"] — the critique.	state["draft"]. The deliverable is the blog post; the critique is metadata about it. Returning the critique means writing a post and then handing back notes on it.
Nothing terminated	No edge to END, and once the loop exists, no stop condition.	Two budgets (section 8) plus explicit END routing.

3. Graph topology

Four nodes. Solid arrows are dispatches from the supervisor; dashed arrows are specialists reporting back. Every specialist returns to the router, which is what makes it a supervisor pattern rather than a chain.

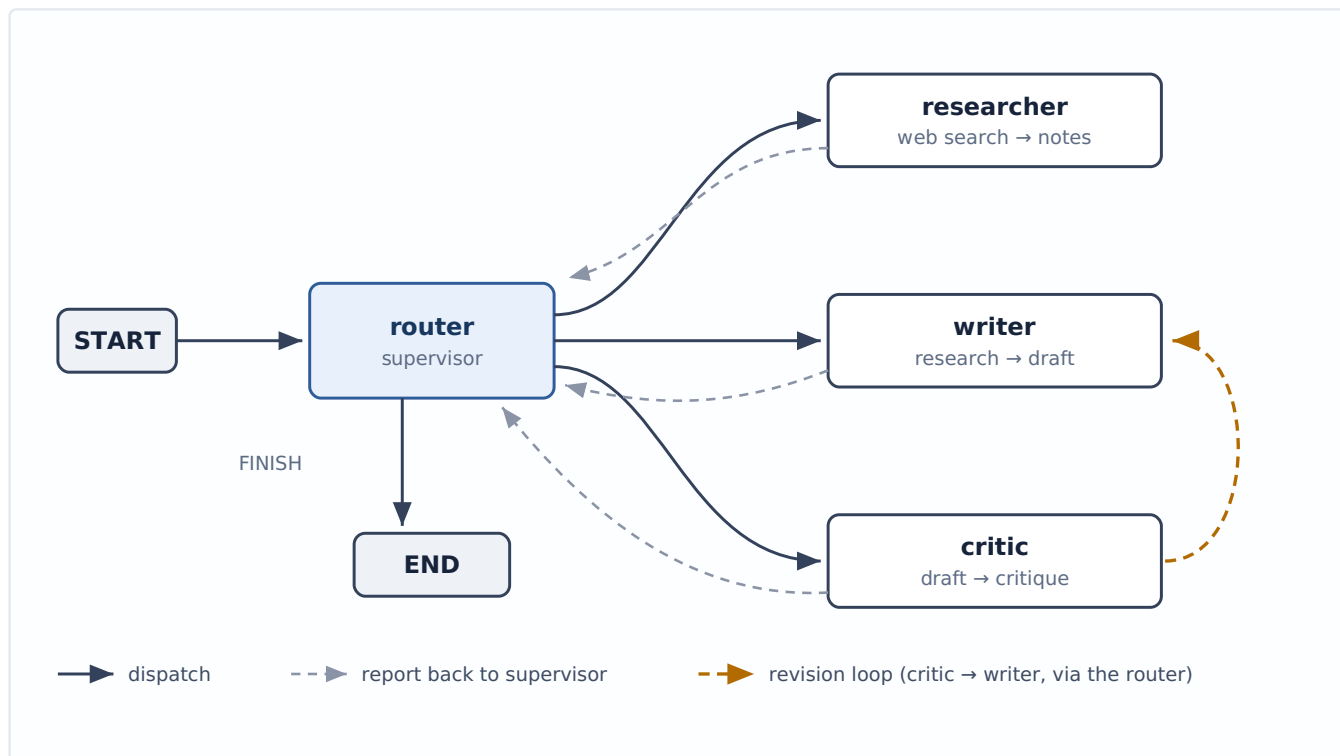


Figure 1 — the compiled graph. The amber loop is the revision cycle: the only cycle that produces new content, and the only one the revision budget bounds.

Note that the router has no `add_conditional_edges` call. It returns a `Command(goto=...)`, and its *return type annotation* is what tells LangGraph which destinations are reachable:

the annotation is load-bearing, not decoration

```
def router(
    state: AgentState, config: RunnableConfig
) -> Command[Literal["researcher", "writer", "critic", "__end__"]]:
```

4. The shared state object

4.1 Field-by-field

AgentState

```
class AgentState(TypedDict):
    """The single object every node reads from and writes into.

    LangGraph hands each node the WHOLE state and merges the partial dict the
    node returns. Edges do not carry selected fields – 'router passes topic to
    researcher' is really 'researcher reads state["topic"]' itself.
    """
    topic: str

    # Shared transcript – the actual agent-to-agent channel.
    # add_messages appends instead of overwriting, so nothing is lost.
    messages: Annotated[list, add_messages]

    # Artifacts produced by each specialist (last-write-wins, single writer each)
    research: str
    sources: list[str]
    draft: str
    critique: str

    # Control state – not content. This is what makes the cycles finite.
    revisions: int # how many drafts the writer has produced
    critique_of_revision: int # which draft the current critique refers to
    steps: int # how many times the supervisor has dispatched

    # Audit trail of routing decisions. operator.add concatenates the lists.
    route_log: Annotated[list[str], operator.add]
```

Field	Written by	Purpose
topic	caller	The task. Read by all three specialists.
messages	all three	Shared transcript — the agent-to-agent channel (section 5).
research	researcher	Condensed factual notes.
sources	researcher	URLs, so the critic can check claims rather than vibe-check prose.
draft	writer	The blog post. This is the final output.
critique	critic	Editorial feedback on the current draft.
revisions	writer	How many drafts exist. Control state, not content.

Field	Written by	Purpose
critique_of_revision	critic	Which draft the critique refers to.
steps	router	How many dispatches have happened.
route_log	all	Audit trail of every routing decision and override.

WHY REVISIONS LIVES IN THE WRITER

Incremented in the writer, `revisions` means “how many drafts exist” — which is what both the budget guard and the router prompt want to reason about. Incremented in the router it would mean “how many times we *intended* to write”, which drifts from reality the moment a guard overrides a route.

WHY CRITIQUE_OF_REVISION IS AN INT, NOT A BOOL

The router must distinguish a critique of the *current* draft from one written about the previous draft. A boolean flag would need clearing by hand in two places and would go stale silently. A revision stamp cannot: `critique_of_revision == revisions` is either true or it isn't.

4.2 Reducers, and why they matter

A reducer decides how a node's returned value is merged into the existing state. Without one, the default is last-write-wins.

the three merge behaviours in this graph

```

messages:  Annotated[list, add_messages]      # appends messages
route_log: Annotated[list[str], operator.add]  # concatenates lists
research:   str                               # no reducer -> overwrite

```

THE PARALLEL-WRITE TRAP

Last-write-wins is correct here *only* because each un-reduced key has exactly one writer. If the researcher and writer ever ran in parallel and both returned the same key, LangGraph raises `InvalidUpdateError` rather than silently picking one. Adding a reducer is how you say what should happen instead.

5. Agent-to-agent message passing

Four scalar strings would be field passing, not message passing. The real channel is `state["messages"]` with the `add_messages` reducer: every specialist appends a *named* `AIMessage`, so each agent — and the router — can read what the previous agents actually said, not just the artifact they left behind.

the researcher's return value – artifact, transcript entry and audit line

```
return {
    "research": notes,
    "sources": sources,
    "messages": [AIMessage(content=f"Research notes:\n{notes}", name="researcher")],
    "route_log": ["researcher: gathered notes from " + str(len(sources)) + " sources"],
}
```

Because `add_messages` appends rather than overwrites, the transcript accumulates across the whole run, including across revisions. This is what the state diagram below shows: each column is one node execution, and each row is a state key.

	router	researcher	router	writer	router	critic	router
topic	set						
research		written					
draft				written			
critique						written	
revisions	0	0	0	1	1	1	1
messages		+1		+1		+1	

 written this step  carried forward, readable by every later node

Figure 2 — state slots filling over a clean run. Darker cells are written on that step; lighter cells are carried forward and readable by every node downstream. The messages row grows monotonically; research, draft and critique are each written by exactly one agent.

6. The supervisor router

6.1 Structured output, not free text

Control flow must never depend on parsing prose. The decision is a Pydantic model, handed to `with_structured_output`, so the set of possible answers is closed by construction:

Route

```
class Route(BaseModel):
    """Structured routing decision – never parse free text for control flow."""
    next: Literal["researcher", "writer", "critic", "FINISH"] = Field(
        description="Which specialist runs next, or FINISH if the work is done."
    )
    reason: str = Field(description="One sentence explaining the choice.")
```

A NOTE ON STRUCTURED OUTPUT AND VALIDATION

As documented in `openai_structured_output.py` in this same folder, structured output *coerces* rather than fails — the model bends its answer to fit the schema. That is exactly what we want for a routing enum (we get a valid destination or nothing), but it is also why the guards in section 7 exist: a syntactically valid route can still be a semantically wrong one.

6.2 The routing prompt

The router is shown what exists so far and what budget remains, and told plainly which decision is genuinely its own:

ROUTER_SYSTEM

```
ROUTER_SYSTEM = """You are the supervisor of a three-agent content team.

Specialists you can dispatch to:
  researcher - runs a web search and produces factual notes on the topic
  writer     - turns the research into a blog post draft; on a rerun it revises
                the existing draft using the critique
  critic     - reviews the current draft against the research for accuracy,
                structure and clarity

Reply FINISH when the draft is good enough to ship.

Your one hard decision is what to do after a critique arrives: send the draft
back to the writer, or FINISH. Weigh whether another rewrite would actually
fix what the critique identifies, and respect the revision budget you are told
about – a draft that is merely imperfect is not worth the last revision."""
```

the per-turn state summary

```
def _router_prompt(state: AgentState, max_revisions: int) -> str:
    """Show the supervisor what exists so far, plus its remaining budget."""
    revisions = state.get("revisions", 0)
    critique_is_current = (
        bool(state.get("critique"))
        and state.get("critique_of_revision", -1) == revisions
    )
    return f"""Topic: {state['topic']}

Current state of the work:
  research : {"DONE (" + str(len(state.get('research', ''))) + " chars)" if state.get("research") else "MISSING"}
  draft    : {"DONE (draft #" + str(revisions) + ")" if state.get("draft") else "MISSING"}
  critique : {"CURRENT (applies to draft #" + str(state.get('critique_of_revision', -1)) + ")" if critique_is_current else ("STALE" if state.get("critique") else "MISSING")}

Revision budget: {revisions} of {max_revisions} drafts used.

Latest critique (empty if none yet):
{state.get("critique", "") or "(none)"}

Which specialist should run next?"""
```


Telling the model its remaining budget changes the question from an open-ended quality judgment (“could this be better?” — always yes) into a cost/benefit one (“is this worth the last revision?”). In practice that produces noticeably less dithering.

7. Routing guards

7.1 Why guards, and what they may not do

The guards enforce *soundness*, not preference. They exist to stop the graph doing something incoherent — entering a specialist without its inputs, or looping forever. They deliberately do **not** second-guess the revise-versus-ship judgment, because that judgment is the router's entire reason for being an LLM.

`_guard_once` – one pass of the four guard rules

```
def _guard_once(
    nxt: str, state: AgentState, max_revisions: int
) -> tuple[str, str | None]:
    """One pass of the routing guards. Returns (corrected_next, reason_or_None).

    The guards enforce soundness, not preference. They never second-guess the
    revise-vs-ship judgment; they only stop the graph doing something incoherent
    (entering a specialist without its inputs, or looping forever).

    Note the writer cap is surgical: it blocks the WRITER specifically rather
    than ending the run outright, so hitting the budget can't ship a draft the
    critic never saw.
    """
    revisions = state.get("revisions", 0)
    critique_is_current = (
        bool(state.get("critique"))
        and state.get("critique_of_revision", -1) == revisions
    )

    # 1. Prerequisites: no writing without research, no critiquing without a draft.
    if nxt == "writer" and not state.get("research"):
        return "researcher", "writer needs research first"
    if nxt == "critic" and not state.get("draft"):
        return "writer", "critic needs a draft first"

    # 2. No re-reviewing an unchanged draft. Without this a router that fixates
    #    on "critic" re-runs it forever: the revision cap bounds the writer, and
    #    a critic that never triggers a rewrite never touches that budget.
    if nxt == "critic" and critique_is_current:
        if revisions < max_revisions:
            return "writer", f"draft #{revisions} already reviewed; nothing new to critique"
        return "FINISH", f"draft #{revisions} already reviewed and budget spent"

    # 3. Revision budget: the writer may not run past the cap.
    if nxt == "writer" and revisions >= max_revisions:
        if not critique_is_current:
            return "critic", f"revision budget spent ({revisions}/{max_revisions}); reviewing instead"
        return "FINISH", f"revision budget spent ({revisions}/{max_revisions})"

    # 4. Never finish on nothing, or on an unreviewed draft.
    if nxt == "FINISH":
        if not state.get("draft"):
            return ("researcher" if not state.get("research") else "writer"), "nothing to ship yet"
        if not critique_is_current:
            return "critic", "current draft has not been reviewed"

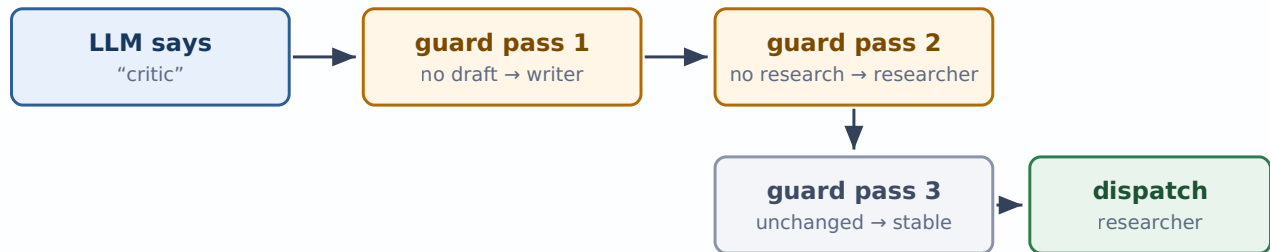
    return nxt, None
```

THE WRITER CAP IS SURGICAL, NOT BLUNT

A naive cap would be `if revisions >= MAX: goto END` at the top of the router. That also blocks legitimate routes: you could hit the cap and end with a draft the critic never reviewed. Guard 3 blocks the *writer* specifically, and falls through to the critic when the current draft is unreviewed.

7.2 Fixed-point iteration

A single guard pass is not enough. Correcting `critic` to `writer` produces a route that may itself need correcting, if no research exists yet. This was the second bug the tests caught: the writer ran with an empty `research` field.



One pass is not enough: correcting a route can produce a route that itself needs correcting. The guards iterate until the answer stops changing, then dispatch. Capped at 4 passes.

Figure 3 — the guards iterate until the route stops changing. The reported reason is the reason for the first correction, since that is the one that explains the LLM's actual mistake.

`_apply_guards`

```
def _apply_guards(
    nxt: str, state: AgentState, max_revisions: int
) -> tuple[str, str | None]:
    """Apply the guards to a fixed point.

    One pass is not enough: correcting 'critic' to 'writer' produces a route
    that may itself need correcting ('writer' with no research yet). Iterate
    until the route stops changing, and report the reason for the FIRST
    correction — that is the one that explains the LLM's actual mistake.
    """
    first_reason = None
    seen = {nxt}
    for _ in range(4): # 4 specialists; a stable route is reached well inside this
        corrected, reason = _guard_once(nxt, state, max_revisions)
        if corrected == nxt:
            return nxt, first_reason
        if first_reason is None:
            first_reason = reason
        nxt = corrected
        # Defensive: two guards disagreeing would ping-pong. Stop and let the
        # whole-graph step budget handle it rather than spinning here.
        if nxt in seen:
            return nxt, first_reason
        seen.add(nxt)
    return nxt, first_reason
```

The `seen` set is defensive: two guard rules that disagreed would ping-pong forever. Rather than spin inside the guard function, it bails out and lets the whole-graph step budget handle it.

8. Termination: two budgets

8.1 The revision budget

Once the critic can route back to the writer, the graph has a cycle, and nothing in that cycle is guaranteed to converge. A critic asked “is this good enough?” can almost always find something to improve, so the honest answer is often “one more pass”, forever. `MAX_REVISIONS` makes the loop finite regardless of what the LLM says.

the two constants

```
MAX_REVISIONS = 2

def max_steps_for(max_revisions: int) -> int:
    return 3 * max_revisions + 4
```

Both are tunable per run without editing code, via the `RunnableConfig` — the same mechanism you would use later for swapping models or toggling verbosity:

```
app.invoke(initial_state, config={"configurable": {"max_revisions": 4}})
```

8.2 The critic loop — a bug the tests caught

The first version of this file had `MAX_REVISIONS` and nothing else. Test case 5 drove the router to answer “critic” every single time, and the graph span until `recursion_limit` killed it:

the failure, before the fix

```
=== 5. router always says 'critic' ===
router -> writer [OVERRIDE 'critic': critic needs a draft first]
router -> critic (lazy)
router -> critic (lazy)
router -> critic (lazy)
... 20 more ...
RAISED GraphRecursionError: Recursion limit of 50 reached without hitting a stop condition.
```

The reason is worth stating precisely, because it generalises:

A PER-SPECIALIST BUDGET ONLY BOUNDS THE SPECIALIST IT NAMES

`MAX_REVISIONS` bounds the *writer*. A router fixated on the critic never triggers a rewrite, so it never touches that budget, and re-reviews the same unchanged draft forever.

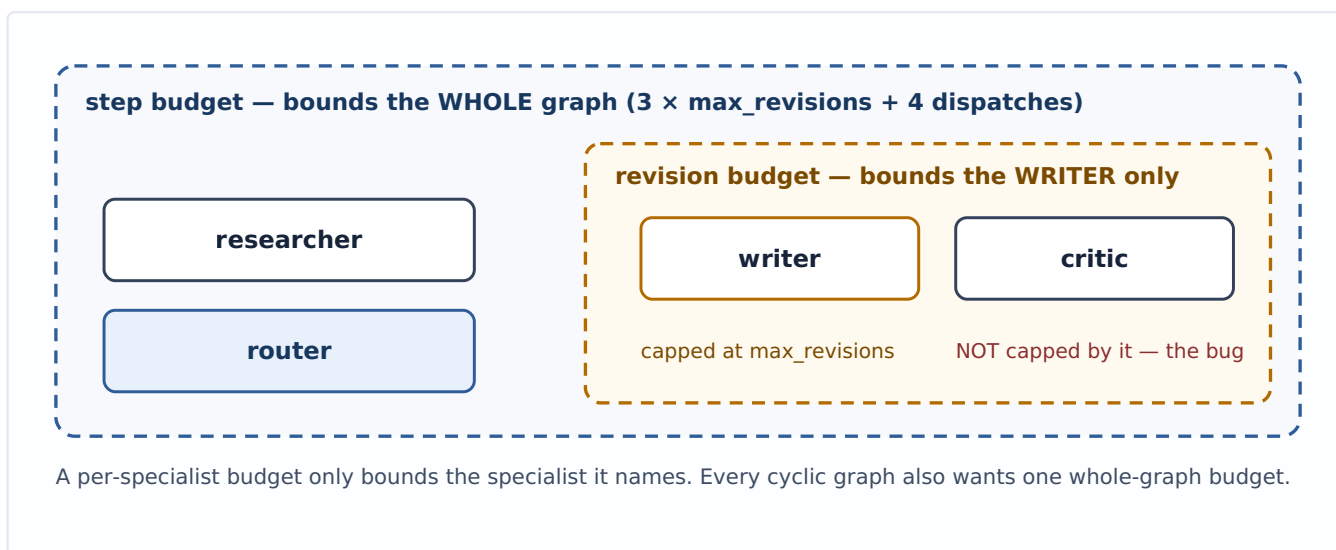


Figure 4 — the revision budget is an inner bound around the writer. The step budget is an outer bound around the whole graph, and it is what catches the failure modes you did not anticipate.

It is fixed twice over. A specific guard, because re-reviewing an unchanged draft is meaningless work regardless of budgets:

the specific fix

```
# 2. No re-reviewing an unchanged draft.
if nxt == "critic" and critique_is_current:
    if revisions < max_revisions:
        return "writer", f"draft #{revisions} already reviewed; nothing new to critique"
    return "FINISH", f"draft #{revisions} already reviewed and budget spent"
```

And a general one, checked *before* the LLM call — a router that has already burned its dispatches does not get another opinion:

the general fix

```
steps = state.get("steps", 0)
if steps >= max_steps:
    log = f"router -> END [STEP BUDGET spent ({steps}/{max_steps})]"
    print(f" {log}")
    return Command(goto=END, update={"route_log": [log]})
```

8.3 recursion_limit is not termination logic

LangGraph's `recursion_limit` is a backstop for bugs, not a design. When it fires you get a `GraphRecursionError` and lose the run entirely; a budget gives you a *deliberate* stop with a usable draft in hand. It is still worth doing the arithmetic once: a full run with 2 revisions is roughly 13 supersteps against a default limit of 25 — comfortable at 2, tight at 3 or 4. Hence `RECURSION_LIMIT = 50`.

9. The three specialists

Each specialist is an ordinary function: read the state it needs, do one job, return a partial update. None of them knows about the others; all coordination is the router's job.

the shared tool – real DuckDuckGo search, no API key

```
def web_search(query: str, max_results: int = 5) -> tuple[str, list[str]]:
    """Real web search (DuckDuckGo, no api key required).

    Returns (formatted_findings, source_urls) so the critic can check claims
    against sources instead of only vibe-checking the prose.
    """
    try:
        results = list(DDGS().text(query, max_results=max_results))
    except Exception as e:
        raise RuntimeError(f"web_search failed: {e}")

    if not results:
        return "No results found", []

    findings = "\n".join(f"- {r['title']}: {r['body']}" for r in results)
    sources = [r["href"] for r in results if r.get("href")]
    return findings, sources
```

specialist 1 – researcher

```
def researcher(state: AgentState) -> AgentState:
    """Web search -> factual notes. Writes research + sources."""
    findings, sources = web_search(state["topic"])

    notes = llm.invoke(
        [
            {
                "role": "system",
                "content": "You are a research assistant. Condense the search results "
                "into tight factual notes a writer can build on. Bullet points, "
                "concrete facts and numbers only, no filler, no opinions.",
            },
            {
                "role": "user",
                "content": f"Topic: {state['topic']}\n\nSearch results:\n{findings}",
            },
        ],
    ).content

    return {
        "research": notes,
        "sources": sources,
        "messages": [AIMessage(content=f"Research notes:\n{notes}", name="researcher")],
        "route_log": ["researcher: gathered notes from " + str(len(sources)) + " sources"],
    }
```

specialist 2 – writer (owns the revisions counter)

```
def writer(state: AgentState) -> AgentState:
    """Research (+ critique on a rerun) -> draft. Owns the revisions counter."""
    revisions = state.get("revisions", 0)
    is_revision = revisions > 0 and bool(state.get("draft"))

    if is_revision:
        task = (
            f"Revise the draft below to address every point in the critique. "
            f"Keep what already works.\n\n"
            f"CURRENT DRAFT:\n{state['draft']}\n\n"
            f"CRITIQUE TO ADDRESS:\n{state['critique']}"
        )
    else:
        task = "Write the first draft of the blog post."

    draft = llm.invoke(
        [
            {
                "role": "system",
                "content": "You are a technical blog writer. Produce a clear, well-structured "
                "post of roughly 400-500 words grounded strictly in the research notes. "
                "Do not invent facts. Output the post only – no preamble.",
            },
            {
                "role": "user",
                "content": f"Topic: {state['topic']}\n\n"
                f"RESEARCH NOTES:\n{state['research']}\n\n{task}",
            },
        ]
    ).content

    label = "revised draft" if is_revision else "first draft"
    return {
        "draft": draft,
        "revisions": revisions + 1,
        "messages": [AIMessage(content=f"Produced {label}.", name="writer")],
        "route_log": [f"writer: produced draft #{revisions + 1} ({label})"],
    }
```

specialist 3 – critic (stamps which draft it reviewed)

```
def critic(state: AgentState) -> AgentState:
    """Draft + research -> critique. Stamps which draft it reviewed."""
    critique = llm.invoke(
        [
            {
                "role": "system",
                "content": "You are a demanding editor. Review the draft against the research "
                "notes for factual accuracy, structure, and clarity. Give at most four "
                "specific, actionable points. If the draft is genuinely ready to publish, "
                "say so plainly and explain why in one sentence.",
            },
            {
                "role": "user",
                "content": f"Topic: {state['topic']}\n\n"
                f"RESEARCH NOTES:\n{state['research']}\n\n"
                f"DRAFT TO REVIEW:\n{state['draft']}",
            },
        ]
    ).content

    revisions = state.get("revisions", 0)
    return {
        "critique": critique,
        # Stamping the revision number is how the router tells a fresh critique
        # from one written about an older draft.
        "critique_of_revision": revisions,
        "messages": [AIMessage(content=f"Critique of draft #{revisions}:\n{critique}", name="critic")],
        "route_log": [f"critic: reviewed draft #{revisions}"],
    }
```

10. Assembling and running the graph

build_graph

```
def build_graph():
    graph = StateGraph(AgentState)
    graph.add_node("router", router)
    graph.add_node("researcher", researcher)
    graph.add_node("writer", writer)
    graph.add_node("critic", critic)

    graph.add_edge(START, "router")

    # Every specialist reports back to the supervisor. The router itself has no
    # add_conditional_edges call – it returns Command(goto=...), and its return
    # type annotation is what tells LangGraph the reachable destinations.
    graph.add_edge("researcher", "router")
    graph.add_edge("writer", "router")
    graph.add_edge("critic", "router")

    return graph.compile()
```


the complete router node

```
def router(
    state: AgentState, config: RunnableConfig
) -> Command[Literal["researcher", "writer", "critic", "__end__"]]:
    """Supervisor node: one LLM call, then the deterministic guards."""
    # Tunable per run without editing code:
    # app.invoke(..., config={"configurable": {"max_revisions": 3}})
    max_revisions = (config.get("configurable") or {}).get("max_revisions", MAX_REVISIONS)
    max_steps = (config.get("configurable") or {}).get("max_steps", max_steps_for(max_revisions))

    # Whole-graph budget, checked BEFORE the LLM call – a router that has already
    # burned its dispatches doesn't get another opinion. The per-specialist guards
    # below handle the failures we can name; this catches the ones we can't.
    steps = state.get("steps", 0)
    if steps >= max_steps:
        log = f"router -> END [STEP BUDGET spent ({steps}/{max_steps})]"
        print(f" {log}")
        return Command(goto=END, update={"route_log": [log]})

    decision: Route = llm.with_structured_output(Route).invoke(
        [
            {"role": "system", "content": ROUTER_SYSTEM},
            {"role": "user", "content": _router_prompt(state, max_revisions)},
        ]
    )

    nxt, override = _apply_guards(decision.next, state, max_revisions)

    log = f"router -> {nxt} ({decision.reason})"
    if override:
        log = f"router -> {nxt} [OVERRIDE '{decision.next}': {override}]"
    print(f" {log}")

    return Command(
        goto=END if nxt == "FINISH" else nxt,
        update={"steps": steps + 1, "route_log": [log]},
    )
```

11. Test results

The graph is tested with a stubbed LLM — no API calls, no network — which makes it possible to script adversarial routers that a real model would rarely produce, and to run the whole suite in under a second. All seven cases pass:

#	Scenario	Asserted behaviour
1	Sensible router, critic satisfied	One draft, reviewed, shipped; all three agents in the transcript
2	Router always says writer	Stops at 2 drafts with a reviewed draft #2
3	Same, max_revisions=4 via config	Stops at 4 drafts — budget is tunable at invoke time
4	Router always says FINISH	Guards force researcher → writer → critic before ending

#	Scenario	Asserted behaviour
5	Router always says critic	Terminates cleanly instead of looping on an unchanged draft
5b	Router always says researcher	Step budget stops it at 10 dispatches
6	writer requested before any research	Redirected to the researcher first

selected output from the wiring test

```

=== 2. router always says 'writer' (cap must stop it) ===
router -> researcher [OVERRIDE 'writer': writer needs research first]
router -> writer (always rewrite)
router -> writer (always rewrite)
router -> critic [OVERRIDE 'writer': revision budget spent (2/2); reviewing instead]
router -> FINISH [OVERRIDE 'writer': revision budget spent (2/2)]
revisions=2 steps=5 draft='BLOG POST BODY' critique_of=2 msgs=5
PASS: stopped at MAX_REVISIONS=2 with a reviewed draft #2

=== 5. router always says 'critic' (the critic loop) ===
router -> researcher [OVERRIDE 'critic': critic needs a draft first]
router -> writer [OVERRIDE 'critic': critic needs a draft first]
router -> critic (lazy)
router -> writer [OVERRIDE 'critic': draft #1 already reviewed; nothing new to critique]
router -> critic (lazy)
router -> FINISH [OVERRIDE 'critic': draft #2 already reviewed and budget spent]
PASS: terminated cleanly instead of looping on an unchanged draft

```

WHY THE ROUTE_LOG IS WORTH KEEPING

Every line above comes from `state["route_log"]`. When a supervisor graph misbehaves in production, the question is always “why did it go there?” — and an audit trail that records both the LLM's choice and any override answers it directly.

12. How to run it

terminal

```

cd ~/Documents/Code_Projects/AI_study/learning/codes

# the wiring test - no API key needed, no network
python langgraph_supervisor_multi_agent_wiring_test.py

# the real thing - needs OPENAI_API_KEY in .env, and network for the search
python langgraph_supervisor_multi_agent.py
python langgraph_supervisor_multi_agent.py "your topic here"

```

Both dependencies are already in this project's `pyproject.toml` (`langgraph` $\geq$ 0.6.11, `ddgs` $\geq$ 9.8.0, `langchain-openai` $\geq$ 0.3.35), so no new installs are required.

The real run prints the routing trace, the agent-to-agent transcript, the final blog post, the final critique, and the source URLs the researcher used.

13. References

- LangGraph — Graph API and state: https://langchain-ai.github.io/langgraph/concepts/low_level/
- LangGraph — Multi-agent supervisor pattern: https://langchain-ai.github.io/langgraph/concepts/multi_agent/
- LangGraph — `Command` for combined state update and routing: <https://langchain-ai.github.io/langgraph/how-tos/command/>
- LangGraph — `GraphRecursionError` troubleshooting: https://python.langchain.com/docs/troubleshooting/errors/GRAPH_RECURSION_LIMIT
- LangChain — `with_structured_output`: https://python.langchain.com/docs/how_to/structured_output/
- ddgs (DuckDuckGo search, no API key): <https://pypi.org/project/ddgs/>
- Local precedents in this repo: `langgraph_multi_agent.py` (supervisor sketch), `langgraph_reducer.py` (add_messages), `langgraph_capstone.py` (Command and conditional edges), `agent_reflexion.py` (ddgs web_search, critique-and-retry loop), `openai_structured_output.py` (structured output caveats)